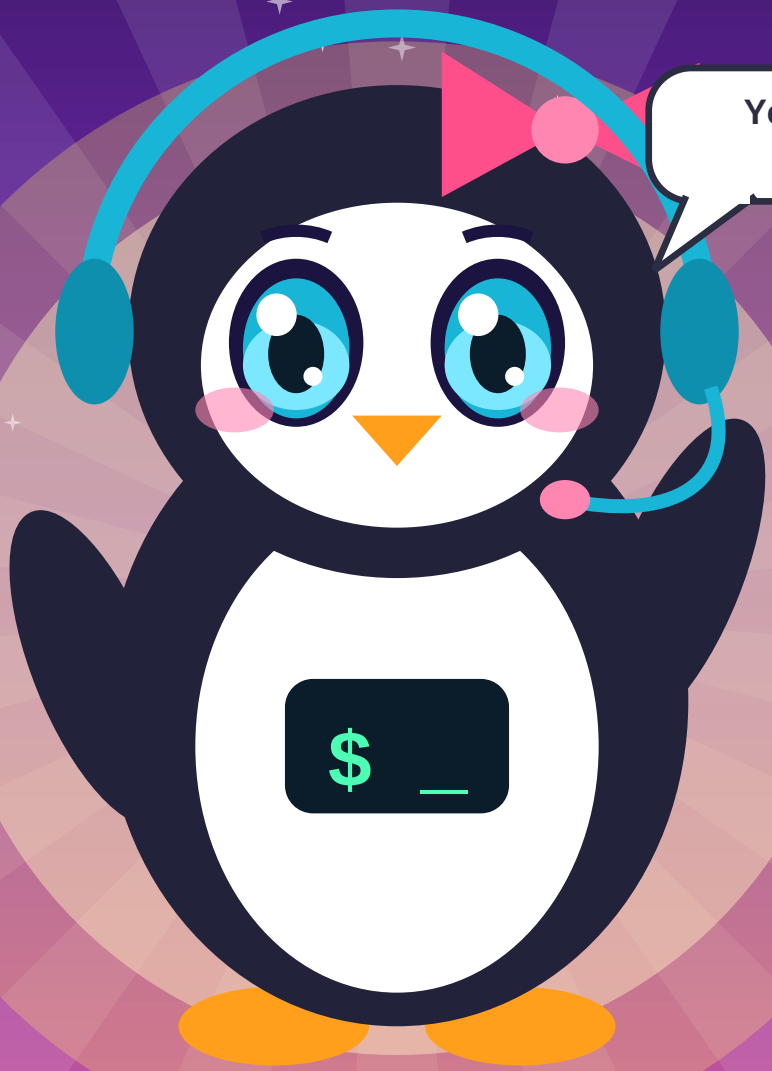
INTERVIEW BOSS BATTLES III

20 NEW SCENARIOS | NETWORKING | SECURITY | K8S | SRE



ULTIMATE BOSS: THE PRINCIPAL ENGINEER HP 100%

- CREATED BY -
RUTUJA NAIK

SET 3 * 20 SCENARIOS * 10 RAPID-FIRE

00 HOW TO ANSWER

TRACE plus layer-by-layer thinking

T	**Triage** How bad is it, who is affected, since when, and what changed recently?
R	**Read the evidence** Check logs, metrics and state before touching anything.
A	**Analyze by layer** Narrow it down: CPU, memory, disk, network, then application and dependencies.
C	**Change safely** Smallest fix first, one change at a time, with a rollback ready.
E	**Educate and prevent** Monitoring, automation, runbook and a blameless post-mortem.

YOUR 60-90 SECOND ANSWER TEMPLATE

1. Restate the problem and the impact (10 s). **2.** Say your order of hypotheses (10 s). **3.** Walk through 3 to 5 commands and what each tells you (40 s). **4.** State the fix and your rollback plan (15 s). **5.** Finish with prevention: monitoring, automation, documentation (10 s).

POWER PHRASES THAT SOUND SENIOR

"Let me walk through it layer by layer."
"First I'd rule out the simplest cause, then go deeper."
"I'd verify the fix with a test, then prevent a repeat."
"If I were on call, I'd mitigate first and communicate often."
"I don't know that one offhand, but here is how I would find out."

QUEST LOG (CONTENTS)

01	Port open locally, closed outside	2	11	HPA is not scaling the pods	12
02	Network config lost after reboot	3	12	Kubernetes Forbidden (RBAC) error	13
03	Too many open files errors	4	13	Pod cannot mount its volume	14
04	Server won't boot after kernel update	5	14	Ingress returns 404 or wrong backend	15
05	RAID degraded and failing disk	6	15	Terraform wants to destroy the DB	16
06	Website is slow but server looks fine	7	16	Ansible playbook failed on some hosts	17
07	Important files deleted by accident	8	17	Recover from a bad git force-push	18
08	Who changed this config file?	9	18	Too many noisy alerts	19
09	Hardening Docker containers	10	19	On-call at 3 AM: incident response	20
10	Over-permissive IAM and old keys	11	20	Database failover and replication lag	21

Troubleshooting | **Admin and security** | **DevOps and cloud** | then: rapid-fire round and final checklist

01 PORT OPEN LOCALLY, CLOSED OUTSIDE

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

Your application works with curl on the server itself, but clients on the network cannot connect to it. How do you find the cause?

SAY IT LIKE THIS (60-90 seconds, out loud)

When it works locally but not remotely, either the app is bound to the wrong address, or something in the path is blocking it: the host firewall, the cloud firewall, or routing. First I check what address the service listens on with `ss -tulpn`. If it shows `127.0.0.1:8080`, it only accepts local connections, and the fix is to bind to `0.0.0.0` or the right interface. If it is bound correctly, I test from another machine with `nc -zv` and read the failure: refused means the host answered but nothing accepted, a timeout means packets are dropped. Then I walk through the layers: the host firewall (ufw, firewalld, iptables or nftables), the cloud security group and NACL, route tables, and any load balancer or NAT in between. For non-standard ports on RHEL I also check SELinux port labels.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`ss -tulpn | grep :8080`	Local address: 127.0.0.1 means local only. 0.0.0.0 or [::] means all interfaces.
2	`curl -v http://SERVER_IP:8080`	From another machine: refused, timed out, or answered?
3	`nc -zv SERVER_IP 8080`	Quick TCP port test with a clear result.
4	`sudo ufw status verbose or sudo firewall-cmd --list-all`	Host firewall rules (Ubuntu or RHEL family).
5	`sudo iptables -S and sudo nft list ruleset`	Underlying rules that may drop or reject the port.
6	Cloud: security group, NACL, route table	Is the port allowed from the client's IP range? Is there a route?
7	`sudo semanage port -l | grep http`	SELinux: non-standard ports need a label (`semanage port -a`).
8	`sudo tcpdump -nn -i any port 8080`	Do SYN packets reach the server? If not, the block is upstream.

COMMON ROOT CAUSES

App bound to 127.0.0.1, host firewall blocking, security group or NACL missing a rule, wrong subnet or route, SELinux blocking a non-standard port, container port not published, load balancer or NAT misconfiguration.

FIX AND PREVENTION

Bind to the right address, open only the needed port for the needed source ranges, then test again from the client. Keep firewall rules in code (Ansible or Terraform) so they are reviewed and repeatable.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. DROP versus REJECT in iptables?

A. REJECT sends an error back immediately, so clients fail fast. DROP silently discards packets, which causes timeouts and hides the host from scanners.

Q. Why use tcpdump here?

A. If the SYN packet reaches the server, the problem is local. If it never arrives, something on the network path blocks it.

AVOID THIS MISTAKE

Opening the port to 0.0.0.0/0 just to test, then forgetting to close it.

02 NETWORK CONFIG LOST AFTER REBOOT

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

You set an IP address and route with the ip command and everything worked. After a reboot the server has no network. Why, and how do you fix it properly?

SAY IT LIKE THIS (60-90 seconds, out loud)

Commands like `ip addr add` and `ip route add` only change the running kernel state, so they disappear on reboot. To make it permanent, the configuration has to live in whatever network manager the distribution uses: netplan with systemd-networkd or NetworkManager on Ubuntu, NetworkManager with `nmcli` on RHEL-family systems, or `/etc/network/interfaces` on older Debian. After a reboot with no network, I use the console and see which tool manages the interface. I check the interface name too, because it can change, for example from eth0 to ens5. Then I write the persistent config and apply it, using `netplan try` where possible so a bad config rolls back automatically. Finally I verify addresses, routes and DNS, and reboot once more to prove it.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`ip -br a and ip route`	Current addresses and routes. Is the interface up with an IP?
2	`nmcli device status or networkctl status`	Which tool manages the interface?
3	`cat /etc/netplan/*.yaml`	Ubuntu: persistent config. Check interface name, addresses and gateway.
4	`sudo netplan try`	Apply with automatic rollback if connectivity is lost. Then `netplan apply`.
5	`nmcli con mod ens5 ipv4.addresses 10.0.1.20/24 ipv4.gateway 10.0.1.1 ipv4.method manual`	NetworkManager: write persistent settings. Then `nmcli con up ens5`.
6	`journalctl -u systemd-networkd -b or -u NetworkManager -b`	Why did the interface fail to come up at boot?
7	`ip link and dmesg | grep -i eth`	Did the interface name or driver change? Is the link up?
8	`resolvectl status`	Confirm DNS servers are set as well, not only IP and route.

COMMON ROOT CAUSES

Manual `ip` changes never persisted, interface renamed (predictable names), YAML indentation errors in netplan, two network managers fighting each other, cloud-init regenerating the config on boot, wrong gateway or subnet mask.

FIX AND PREVENTION

Define settings in the managed config, use `netplan try` or an out-of-band console when changing remote servers, and automate with cloud-init or Ansible. If you maintain config by hand on a cloud-init host, disable cloud-init's network regeneration.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Why did eth0 become ens5?

A. Predictable interface names come from hardware or bus information, so names can differ between machines and instance types.

Q. What does netplan try do?

A. It applies the config and waits for confirmation. If you don't confirm in time, it reverts, so you don't lock yourself out.

AVOID THIS MISTAKE

Changing the network on a remote server without a rollback or console access.

03 TOO MANY OPEN FILES ERRORS

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

An application intermittently fails with 'Too many open files'. How do you diagnose and fix it?

SAY IT LIKE THIS (60-90 seconds, out loud)

Every file, socket and pipe uses a file descriptor, and each process has a limit. First I see whether the problem is the per-process limit or the system-wide one. I read the process's real limit from `/proc/PID/limits` and count its open descriptors with `ls /proc/PID/fd | wc -l` or `lsof -p`. A steadily growing count means a leak: connections or files not being closed. A count close to the limit under normal load means the limit is simply too low. For services under systemd, the shell's `ulimit` doesn't apply, so I set `LimitNOFILE` in the unit. I'd also check the system-wide `fs.file-max`, and fix the leak in code if there is one.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`grep 'open files' /proc/PID/limits`	Soft and hard limit for the running process.
2	`ls /proc/PID/fd | wc -l`	How many descriptors it has open right now.
3	`lsof -p PID | awk '{print $5}' | sort | uniq -c`	Breakdown by type: REG files, IPv4 sockets, pipes. Shows what leaks.
4	`ss -tan | awk '{print $1}' | sort | uniq -c`	TCP states. Many CLOSE_WAIT means the app isn't closing sockets.
5	`systemctl show app -p LimitNOFILE`	The limit systemd applies to the service.
6	`sudo systemctl edit app (add LimitNOFILE=65536)`	Raise the limit for a systemd service, then restart it.
7	`cat /proc/sys/fs/file-max and cat /proc/sys/fs/file-nr`	System-wide maximum and current usage.
8	`ulimit -n and /etc/security/limits.conf`	Shell and PAM limits for login sessions (not used by systemd services).

COMMON ROOT CAUSES

File or socket leak in the application, default limit of 1024 too low for a busy server, a connection storm, many CLOSE_WAIT sockets from unclosed connections, limits set in the wrong place (the shell instead of the unit), huge numbers of log or temp files.

FIX AND PREVENTION

Raise `LimitNOFILE` to a sensible value, fix leaks (close in finally blocks or use connection pools), set connection timeouts, and monitor descriptor use per process. Alert at 80% of the limit.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Soft versus hard limit?

A. The soft limit is what the process currently has. It may raise it up to the hard limit. Only root can raise the hard limit.

Q. What does CLOSE_WAIT indicate?

A. The remote side closed the connection, but the local application hasn't closed its end yet. Usually an application bug.

AVOID THIS MISTAKE

Only raising the limit and never checking whether descriptors keep growing.

04 SERVER WON'T BOOT AFTER KERNEL UPDATE

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

After a kernel update and reboot, a server hangs or panics during boot. How do you recover?

SAY IT LIKE THIS (60-90 seconds, out loud)

The advantage on Linux is that old kernels stay installed, so I boot the previous one from the GRUB advanced options. On a cloud VM I use the serial console, since GRUB isn't reachable over the network. Once the system is up on the old kernel, I make it the default to restore service, then investigate. I read the boot errors with `journalctl -b -1`, and check whether the initramfs was built correctly, whether a DKMS driver module failed to build for the new kernel, or whether the root device or filesystem driver is missing. The fix is usually rebuilding the initramfs and modules, or removing the bad kernel and pinning a known-good version until the cause is clear. In future I'd snapshot the VM before kernel updates.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	GRUB menu: Advanced options, choose the previous kernel	Boot the older, known-good kernel first to restore service.
2	`uname -r and dpkg -l | grep linux-image (or rpm -q kernel)`	Which kernel is running, and which are installed?
3	`journalctl -b -1 -p err and dmesg`	Errors from the failed boot: panic, missing root device, module failures.
4	`sudo grubby --set-default /boot/vmlinuz-X.Y (RHEL) or edit GRUB_DEFAULT`	Make the working kernel the default.
5	`sudo dkms status`	Did out-of-tree modules (GPU, storage, VPN) fail to build for the new kernel?
6	`sudo update-initramfs -u -k all or sudo dracut -f --regenerate-all`	Rebuild initramfs images for installed kernels.
7	`sudo apt-mark hold linux-image-generic`	Pin the kernel while you investigate (RHEL: versionlock).
8	`sudo apt remove linux-image-X.Y then sudo update-grub`	Remove the bad kernel and regenerate the GRUB config.

COMMON ROOT CAUSES

Missing or corrupted initramfs, DKMS module failed to compile, storage or filesystem driver absent from the initramfs, wrong root= or UUID in the GRUB config, no free space in /boot during install, incompatible hardware or virtualisation drivers.

FIX AND PREVENTION

Boot the old kernel, rebuild the initramfs and modules, keep /boot with free space, and test kernel updates in staging first. Snapshot VMs before kernel updates and keep at least two kernels installed.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Why keep multiple kernels installed?

A. So you can boot an older kernel if the newest one fails.

Q. What does the initramfs do?

A. A small early filesystem with the drivers and scripts needed to find and mount the real root filesystem.

AVOID THIS MISTAKE

Removing all old kernels before confirming the new one works.

05 RAID DEGRADED AND FAILING DISK

ADMIN & SECURITY DIFFICULTY ✦✦✦

THE QUESTION

Monitoring reports a degraded software RAID array and SMART warnings on one disk. What do you do?

SAY IT LIKE THIS (60-90 seconds, out loud)

A degraded array is still running, but it has lost its redundancy, so a second failure can mean data loss. I confirm the state with `cat /proc/mdstat` and `mdadm --detail` and identify the failed member. I check the SMART data for that disk and the kernel log for IO errors. Before touching anything, I make sure there's a current backup, because a rebuild stresses the remaining disks. Then I mark the disk failed and remove it with mdadm, physically replace it, copy the partition layout from a good disk, add the new partition to the array, and monitor the resync. I confirm the array returns to a clean state, fix the monitoring and alerts, and check the other disks' health, because disks from the same batch tend to fail at similar times.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`cat /proc/mdstat`	Array status. `[U_]` means one member is missing, `[UU]` means healthy.
2	`sudo mdadm --detail /dev/md0`	State, failed device and rebuild progress.
3	`sudo smartctl -a /dev/sdb`	SMART attributes: reallocated or pending sectors, and the error log.
4	`dmesg -T | grep -iE 'i/o error|ata|md0'`	Kernel IO errors, and which disk they belong to.
5	`sudo mdadm /dev/md0 --fail /dev/sdb1 --remove /dev/sdb1`	Mark the failing member failed and remove it.
6	`sudo sfdisk -d /dev/sda | sudo sfdisk /dev/sdb`	Copy the partition layout from a good disk to the replacement.
7	`sudo mdadm /dev/md0 --add /dev/sdb1`	Add the new member. The rebuild starts automatically.
8	`watch cat /proc/mdstat`	Watch resync progress and the performance impact on the application.

COMMON ROOT CAUSES

Disk wear or age, bad sectors, cable or controller problems, power issues, firmware bugs, and several disks from the same batch ageing together.

FIX AND PREVENTION

Replace the failed disk, let it resync, then check the other disks. Configure `mdadm --monitor` and smartd alerts, keep spare disks, and remember that RAID is not a backup.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. RAID 1 versus RAID 5 versus RAID 10?

A. RAID 1 mirrors (survives one failure). RAID 5 uses parity (survives one failure, slower writes). RAID 10 stripes mirrors: fast, and survives one failure per mirror pair.

Q. Why isn't RAID a backup?

A. It only protects against disk failure. Deletion, corruption, ransomware and fire affect every member.

AVOID THIS MISTAKE

Pulling the wrong disk. Verify the serial number with smartctl before removing hardware.

06 WEBSITE IS SLOW BUT SERVER LOOKS FINE

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

Users say the website is slow, but CPU, memory and disk on the web server look normal. How do you find the bottleneck?

SAY IT LIKE THIS (60-90 seconds, out loud)

If the server's own resources are fine, the delay is somewhere else in the request path. I measure where the time goes using `curl` timing variables: DNS lookup, TCP connect, TLS handshake, time to first byte and total time. That shows whether the delay is network, TLS or the application. If time to first byte is high, the application or its dependencies are slow: the database, an external API, a cache-miss storm or a lock. I check the web server's access log for response times, the application's own metrics or traces, and the slow query log. Then I look at the client side as well: large assets, no compression or caching, no CDN. I fix the biggest contributor first and measure again.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`curl -o /dev/null -s -w 'dns:%{time_namelookup} connect:%{time_connect} tls:%{time_appconnect} ttfb:%{time_starttransfer} total:%{time_total}\n' https://site`	Breaks the request into phases and shows where the delay is.
2	`awk '{print $NF}' access.log | sort -n | tail`	Slowest requests, if the log format includes request time.
3	`ss -s and netstat -s | grep -i retrans`	Connection counts and TCP retransmissions.
4	`mysqladmin processlist or SELECT * FROM pg_stat_activity;`	Are slow or blocked queries piling up?
5	Slow query log and EXPLAIN ANALYZE	Find missing indexes and expensive queries.
6	APM or tracing (OpenTelemetry, Jaeger)	See time per service call and per database query.
7	`curl -I -H 'Accept-Encoding: gzip' https://site`	Are compression and caching enabled (Content-Encoding, Cache-Control)?
8	Browser DevTools, Network tab	Large images, many requests, blocking scripts on the client side.

COMMON ROOT CAUSES

Slow database queries or locks, external API latency, cache misses, connection pool exhaustion, DNS or TLS delays, no compression or caching, large assets, distance without a CDN, thread pool saturation inside the app.

FIX AND PREVENTION

Fix the biggest bottleneck first: add indexes, cache results, tune pools, add a CDN and compression, and optimise assets. Define latency SLOs and dashboards for p50, p95 and p99.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. What is time to first byte?

A. The time from sending the request to receiving the first byte of the response. High TTFB points to slow server-side processing.

Q. Why look at p95 and p99 instead of the average?

A. Averages hide the slow tail. A few very slow requests can affect many users.

AVOID THIS MISTAKE

Adding more servers before you know where the time is spent.

07 IMPORTANT FILES DELETED BY ACCIDENT

ADMIN & SECURITY DIFFICULTY ✦✦✦

THE QUESTION

A colleague ran rm -rf on a directory with important data on a production server. What do you do?

SAY IT LIKE THIS (60-90 seconds, out loud)

First I stop further damage. New writes can overwrite deleted data, so I stop the application writing there, or remount the filesystem read-only, and tell the team not to touch it. Then I look for proper recovery sources: backups, snapshots (LVM, ZFS, Btrfs or cloud volume snapshots), versioned object storage, or the Git repository. A file that was deleted but is still open by a process can be copied back from `/proc/PID/fd`. Only if none of that exists do I try recovery tools, such as `extundelete` or `testdisk` on ext4, ideally working on a copy of the disk, with no guarantees. Afterwards I run a blameless review and add protection: tested backups, snapshots before risky changes, and restricted privileges.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	Stop writes to that filesystem	Stop the app or remount read-only (`mount -o remount,ro /data`) to avoid overwriting freed blocks.
2	`lsof +L1`	Files deleted but still open can be copied back from `/proc/PID/fd/N`.
3	`aws ec2 describe-snapshots --filters Name=volume-id,Values=vol-123`	Look for recent cloud snapshots to restore from.
4	`lvs and zfs list -t snapshot and btrfs subvolume list /`	Local snapshots on LVM, ZFS or Btrfs.
5	`restic snapshots (or your backup tool)`	Restore the last good backup of that path.
6	`git status and git restore .`	If the folder was a Git repository, restore from history.
7	`sudo extundelete /dev/sdb1 --restore-all`	Last resort on unmounted ext3 or ext4. Works best on a copy; no guarantees.
8	`sudo testdisk or photorec`	Other recovery tools. Always write recovered files to a different disk.

KEY POINTS TO MENTION

Freed blocks can be overwritten quickly, so speed and stopping writes matter. Recovery tools are unreliable on busy filesystems and on SSDs with TRIM. Backups and snapshots are the real answer.

FIX AND PREVENTION

Take regular, tested backups and snapshots, snapshot before risky operations, use least privilege, and use `rm -i` or a trash utility in interactive shells. Require review for destructive production commands.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Why is recovery harder on SSDs?

A. TRIM tells the SSD to erase freed blocks, so deleted data may be gone quickly.

Q. Can you recover a file that is deleted but still open?

A. Yes, while the process holds it open: copy it from /proc/PID/fd/N before the process exits.

AVOID THIS MISTAKE

Installing recovery tools onto the same disk. That can overwrite the deleted data.

08 WHO CHANGED THIS CONFIG FILE?

ADMIN & SECURITY DIFFICULTY ✦✦✦

THE QUESTION

A critical configuration file was changed and the service broke. How do you find who changed it, when, and what exactly changed?

SAY IT LIKE THIS (60-90 seconds, out loud)

I answer three questions: what changed, when, and who. For the content, I compare against a known-good copy: version control if the file is managed by Git or configuration management, or a backup. `stat` gives the modification time, and the package manager can verify files it installed, with `rpm -V` or `debsums`. For who, I use auditd if it was enabled: a watch rule on that file records the user, command and time, and I search it with `ausearch`. Without auditd I rely on indirect evidence: `last` for logins around that time, journald entries for `sudo`, and shell history, which is weak evidence. Afterwards I put the file under audit rules and configuration management so every change is traceable.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`stat /etc/app/app.conf`	Last modify and change times: when did it change?
2	`diff /etc/app/app.conf /backup/app.conf or git diff`	What exactly changed compared to a known-good version?
3	`rpm -V package or debsums -c`	Verify installed package files against their checksums.
4	`sudo ausearch -f /etc/app/app.conf -i`	auditd events: user, process, syscall and time for that file.
5	`sudo auditctl -w /etc/app/app.conf -p wa -k appconf`	Add a watch rule for writes and attribute changes, with a key.
6	`sudo journalctl _COMM=sudo --since '2 hours ago'`	Which user ran which sudo command, and when?
7	`last -a | head and w`	Who was logged in around that time, and from where?
8	Config management or Git history	An Ansible run or a Git commit may explain an intended change.

KEY POINTS TO MENTION

Auditd gives a reliable trail only if it was already watching. Shell history is easy to edit, so don't trust it for investigations. Shared admin accounts make attribution impossible, so use named accounts with sudo.

FIX AND PREVENTION

Manage config through Git and automation, restrict write access, enable auditd rules on critical files, ship logs to a central system, and alert on unexpected changes with a file integrity tool such as AIDE or Wazuh.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. What is file integrity monitoring?

A. A tool that records checksums of important files and alerts when they change unexpectedly (AIDE, Tripwire, Wazuh).

Q. Why avoid shared accounts?

A. You can't tell who did what. Named accounts with sudo create an audit trail.

AVOID THIS MISTAKE

Treating bash history as proof. Users can disable or edit it.

09 HARDENING DOCKER CONTAINERS

ADMIN & SECURITY DIFFICULTY ✦✦✦

THE QUESTION

How would you secure a Docker container and the way it runs in production?

SAY IT LIKE THIS (60-90 seconds, out loud)

I secure it in layers: the image, the container at runtime, and the host. For the image, I use a minimal trusted base, multi-stage builds, pinned versions, a Trivy scan in CI, and no secrets in layers. At runtime, I run as a non-root user, use a read-only root filesystem with `tmpfs` where writes are needed, drop all Linux capabilities and add back only what's required, set `no-new-privileges`, apply memory, CPU and PID limits, and never use `--privileged` or mount the Docker socket. For networking, I publish only the needed ports and use user-defined networks. On the host, I keep Docker and the kernel patched, use a hardened OS, enable rootless mode or user namespaces where possible, and send logs centrally.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`trivy image app:1.0`	Scan an image for known vulnerabilities before it ships.
2	`USER 10001 (in the Dockerfile)`	Run as a non-root user by default.
3	`docker run --read-only --tmpfs /tmp app`	Immutable filesystem. Only /tmp is writable.
4	`docker run --cap-drop ALL --cap-add NET_BIND_SERVICE app`	Drop every capability and add back only the one needed.
5	`docker run --security-opt no-new-privileges app`	Block privilege escalation through setuid binaries.
6	`docker run --memory 512m --cpus 1 --pids-limit 200 app`	Resource limits reduce the damage from abuse.
7	`docker inspect -f '{{.HostConfig.Privileged}}' c`	Confirm nothing runs privileged. Also avoid mounting /var/run/docker.sock.
8	`hadolint Dockerfile and a CIS Docker Benchmark check`	Lint the Dockerfile and audit the daemon and host settings.

KEY POINTS TO MENTION

A container shares the host kernel, so it is not a boundary like a VM. Least privilege, minimal images and patching reduce risk. Mounting the Docker socket or running privileged is effectively root on the host.

FIX AND PREVENTION

Bake these controls into CI (image scanning, policy checks) and runtime policy (Kubernetes Pod Security Standards, seccomp and AppArmor profiles). Sign images and pull only from trusted registries.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Why is mounting docker.sock dangerous?

A. Whoever controls the socket can start containers with host access, which equals root on the host.

Q. What is rootless Docker?

A. Running the Docker daemon and containers without root privileges, so a breakout has fewer rights.

AVOID THIS MISTAKE

Running containers as root with --privileged because it just works.

10 OVER-PERMISSIVE IAM AND OLD KEYS

ADMIN & SECURITY DIFFICULTY ✦✦✦

THE QUESTION

A security review finds IAM users with admin rights and access keys that are two years old. How do you fix this without breaking things?

SAY IT LIKE THIS (60-90 seconds, out loud)

I treat it as a risk-reduction project, not a switch to flip. First, discovery: the IAM credential report lists users and key ages, and the 'last used' data and Access Analyzer show which permissions are really used. Then I prioritise: deactivate unused keys and users, enforce MFA, and replace long-lived keys with roles: instance profiles, IRSA for Kubernetes and OIDC for CI. For permissions, I follow least privilege: build policies from actual CloudTrail activity, separate admin roles from daily-use roles, and add permission boundaries and SCPs as guardrails. To avoid breakage, I deactivate before deleting, watch CloudTrail for denials, and keep a rollback path. Then I automate rotation, alerts and periodic reviews.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`aws iam generate-credential-report; aws iam get-credential-report`	All users with password and access key age and last-used dates.
2	`aws iam list-access-keys --user-name u`	Keys per user, with status and creation date.
3	`aws iam get-access-key-last-used --access-key-id AKIA...`	Is the key used at all, and by which service?
4	`aws iam update-access-key --access-key-id AKIA... --status Inactive`	Deactivate first, observe, and delete after a quiet period.
5	`aws accessanalyzer list-findings --analyzer-arn ARN`	Find unused permissions and unintended external access.
6	`aws cloudtrail lookup-events --lookup-attributes AttributeKey=Username,AttributeValue=u`	What did this identity actually do?
7	`aws iam list-mfa-devices --user-name u`	Is MFA enabled? Require it for console and sensitive actions.
8	**Replace keys with roles**	EC2 instance profiles, IRSA for EKS, GitHub OIDC for CI, SSO for humans.

COMMON ROOT CAUSES

Admin policies granted for convenience, shared long-lived keys in scripts or CI, no key rotation, no MFA, no offboarding process, and unused users and roles accumulating.

FIX AND PREVENTION

Use SSO and roles with temporary credentials, enforce MFA, rotate or remove keys, apply least privilege from real usage data, add SCP guardrails, and review access regularly with automated reports and alerts.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Why prefer roles over access keys?
A. Roles issue short-lived credentials automatically, so there is nothing long-lived to leak or rotate.

Q. What is a permissions boundary?
A. A policy that sets the maximum permissions a user or role can ever have, regardless of other policies attached.

AVOID THIS MISTAKE

Deleting keys immediately. Deactivate first in case something still uses them.

11 HPA IS NOT SCALING THE PODS

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

Traffic has increased and the application is slow, but the Horizontal Pod Autoscaler is not adding pods. How do you troubleshoot it?

SAY IT LIKE THIS (60-90 seconds, out loud)

I start with `kubectl describe hpa`, which shows current and target metrics, conditions and events. The most common problem is that the HPA can't read metrics at all and shows `<unknown>`. That means metrics-server is missing or unhealthy, or the pods have no resource requests, because CPU utilisation is calculated relative to the request. Next I check whether it's already at `maxReplicas`, whether the target makes sense, and whether new pods can actually be scheduled. Pods stuck in Pending mean the cluster needs node capacity, so I check the cluster autoscaler. I also remember that stabilisation windows can delay scaling, and that CPU may not be the real bottleneck at all.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`kubectl get hpa and kubectl describe hpa web`	Current vs target metrics, replica counts, conditions (AbleToScale, ScalingActive) and events.
2	`kubectl top pods and kubectl top nodes`	Does metrics-server work? Errors here mean the HPA is blind.
3	`kubectl get deploy web -o yaml | grep -A6 resources`	CPU utilisation targets need `requests` on the containers.
4	`kubectl -n kube-system get pods | grep metrics-server`	Is metrics-server running and healthy?
5	`kubectl get hpa web -o jsonpath='{.spec.maxReplicas}'`	Is it already pinned at the maximum?
6	`kubectl get pods | grep Pending`	New replicas stuck Pending mean no node capacity. Check the cluster autoscaler.
7	`kubectl describe hpa web | grep -i -A5 behavior`	Stabilisation windows and scaling policies can slow reaction.
8	Check the real bottleneck	If the database or a downstream API is slow, adding pods won't help.

COMMON ROOT CAUSES

metrics-server missing or failing, no CPU or memory requests set, maxReplicas too low, wrong target or metric, pods unschedulable (no node capacity), slow scale-up policies, or the bottleneck is elsewhere (database, downstream service).

FIX AND PREVENTION

Install and monitor metrics-server, set realistic requests, tune min and max replicas and targets, enable the cluster autoscaler, and use custom or request-rate metrics (KEDA or Prometheus adapter) for non-CPU workloads. Load test the scaling behaviour.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. HPA versus VPA versus cluster autoscaler?

A. HPA changes the number of pods. VPA adjusts pod resource requests. The cluster autoscaler adds or removes nodes.

Q. Why does the HPA need resource requests?

A. Utilisation is usage divided by the request. Without a request it cannot calculate a percentage.

AVOID THIS MISTAKE

Raising maxReplicas without checking whether the pods can be scheduled.

12 KUBERNETES FORBIDDEN (RBAC) ERROR

ADMIN & SECURITY DIFFICULTY ✦✦✦

THE QUESTION

A deployment pipeline fails with: Error from server (Forbidden): pods is forbidden: User cannot list resource in namespace. How do you fix it safely?

SAY IT LIKE THIS (60-90 seconds, out loud)

Forbidden means authentication worked but RBAC authorisation denied the request. The error names the user or service account, the verb, the resource and the namespace, so I start with that. I confirm who is calling, from the kubeconfig or the service account, then reproduce the denial with **`kubectl auth can-i`** using impersonation. Then I look at which Roles and RoleBindings or ClusterRoleBindings apply to that subject. The fix is to create a Role with only the verbs and resources the pipeline needs, bind it to that service account in that namespace, and test again. I avoid binding cluster-admin, and I manage RBAC as code in Git.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`kubectl auth can-i list pods -n ns --as=system:serviceaccount:ns:ci`	Test one specific permission for the subject using impersonation.
2	`kubectl auth can-i --list -n ns --as=system:serviceaccount:ns:ci`	Everything that subject is allowed to do.
3	`kubectl get rolebinding,clusterrolebinding -A -o wide | grep ci`	Which bindings reference this subject?
4	`kubectl describe role pipeline -n ns`	Verbs, resources and apiGroups the role grants.
5	`kubectl create role deployer --verb=get,list,watch,create,update,patch --resource=deployments,pods -n ns`	Create a minimal Role.
6	`kubectl create rolebinding ci-deployer --role=deployer --serviceaccount=ns:ci -n ns`	Bind the Role to the service account in that namespace.
7	`kubectl get sa ci -n ns -o yaml`	Confirm the service account exists and is the one the pipeline uses.
8	`kubectl auth can-i create deployments -n ns --as=system:serviceaccount:ns:ci`	Re-test after the fix, and confirm unwanted actions are still denied.

KEY POINTS TO MENTION

RBAC is additive and deny-by-default: a subject has only what bindings grant. Roles are namespaced and ClusterRoles are cluster-wide, but a ClusterRole can be bound inside one namespace with a RoleBinding. Check apiGroups: **`apps`** for deployments, the empty string for core resources like pods.

FIX AND PREVENTION

Grant least privilege per service account, keep RBAC manifests in Git, review bindings regularly, use short-lived tokens (OIDC, projected tokens), and audit with tools such as **`kubectl-who-can`** or **`rbac-lookup`**.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Role versus ClusterRole?

A. A Role grants permissions inside one namespace. A ClusterRole is cluster-wide, or reusable across namespaces through RoleBindings.

Q. 401 versus 403?

A. 401: authentication failed (who are you?). 403: authenticated, but not authorised by RBAC.

AVOID THIS MISTAKE

Fixing it by binding cluster-admin to the pipeline.

13 POD CANNOT MOUNT ITS VOLUME

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

A pod stays in ContainerCreating with the event: Multi-Attach error for volume, volume is already exclusively attached to one node. What is happening, and how do you fix it?

SAY IT LIKE THIS (60-90 seconds, out loud)

This is a storage access mode problem. Block volumes like EBS or Azure Disk are ReadWriteOnce, which means they attach to only one node at a time. When the pod moves to another node, for example during a rolling update or after a node failure, the new pod can't start until the old node releases the volume. So I check where the volume is currently attached and whether the old pod is still terminating, or the old node is unhealthy. For single-replica apps with rolling updates, I switch to the `Recreate` strategy so the old pod stops first. If the node is dead, the volume may stay attached until the cloud detaches it, so I check the attachment state. For real multi-node access I'd use ReadWriteMany storage such as EFS or NFS.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`kubectl describe pod p`	Events show the Multi-Attach or mount error, and which volume is involved.
2	`kubectl get pvc,pv`	PVC status, bound PV, access mode (RWO, RWX) and storage class.
3	`kubectl get pods -o wide`	Where is the old pod running, and where was the new one scheduled?
4	`kubectl get volumeattachment`	Which node the volume is currently attached to.
5	`kubectl get deploy app -o jsonpath='{.spec.strategy}'`	RollingUpdate with an RWO volume can deadlock. Consider Recreate.
6	`aws ec2 describe-volumes --volume-ids vol-123 --query 'Volumes[].Attachments'`	Cloud view: attached to which instance, and in what state?
7	`kubectl describe node NODE`	Is the old node NotReady? That delays the detach.
8	`kubectl get storageclass`	Does the class support RWX or only RWO? Is binding mode WaitForFirstConsumer?

COMMON ROOT CAUSES

ReadWriteOnce volume used with rolling updates or several replicas on different nodes, old node failed or slow to detach, zone mismatch (volume in one AZ, pod in another), stale VolumeAttachment, wrong storage class.

FIX AND PREVENTION

Use the `Recreate` strategy for single-replica apps, StatefulSets with per-pod volumes, RWX storage for shared data, topology-aware scheduling (WaitForFirstConsumer), and handle node failures. Back up volumes with snapshots.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. RWO versus RWX?

A. ReadWriteOnce: read-write by a single node. ReadWriteMany: by many nodes at once (needs NFS, EFS, CephFS or similar).

Q. Why do EBS volumes cause zone problems?

A. An EBS volume lives in a single AZ, so the pod must run on a node in that AZ.

AVOID THIS MISTAKE

Force-detaching the volume while the old pod might still be writing.

14 INGRESS RETURNS 404 OR WRONG BACKEND

TROUBLESHOOTING DIFFICULTY ✦✦✦

THE QUESTION

You deployed a new service and Ingress, but requests return 404 or reach the wrong backend. How do you troubleshoot?

SAY IT LIKE THIS (60-90 seconds, out loud)

I follow the path: DNS, load balancer, ingress controller, ingress rule, Service, pod. First I confirm that the request reaches the ingress controller and which host and path it carries, using `curl -v` with an explicit Host header. A 404 from the controller usually means no rule matched: the host or path doesn't match, the `ingressClassName` doesn't match the controller, or the Ingress is in a different namespace from its Service. Then I check the Ingress object's backends and events, the Service and its endpoints, and the controller's logs, which show the matched upstream and status. I also check path types and rewrite annotations, because a wrong rewrite can send the app a path it doesn't know, producing a 404 from the application itself.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`curl -v -H 'Host: app.example.com' http://INGRESS_IP/path`	Test the exact host and path against the controller's IP.
2	`kubectl get ingress -A and kubectl describe ingress app`	Hosts, paths, backends, class and events (such as sync errors).
3	`kubectl get ingressclass`	Does `ingressClassName` match an installed controller?
4	`kubectl get svc,endpoints app`	Does the backend Service exist in the same namespace, with endpoints?
5	`kubectl -n ingress-nginx logs deploy/ingress-nginx-controller --tail=50`	Controller log: which upstream matched, and what status came back.
6	`kubectl exec -n ingress-nginx POD -- grep -A5 app.example.com /etc/nginx/nginx.conf`	See the generated config for your host (nginx ingress).
7	`kubectl get ingress app -o yaml | grep -E 'pathType|rewrite'`	Prefix versus Exact path types, and rewrite-target annotations.
8	`dig +short app.example.com`	Does DNS point at the right load balancer?

COMMON ROOT CAUSES

Host or path mismatch, missing or wrong ingressClassName, Ingress and Service in different namespaces, Service without endpoints, wrong port name or number, a rewrite annotation sending the wrong path, the default backend responding, DNS pointing elsewhere, missing TLS secret.

FIX AND PREVENTION

Align host, path and class, fix the Service and port, correct rewrite rules, and test with explicit Host headers. Add smoke tests after deployments and keep Ingress manifests in Git.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Ingress versus a LoadBalancer Service?

A. A LoadBalancer Service exposes one service. Ingress routes many hosts and paths to many services through one entry point.

Q. What is the Gateway API?

A. A newer, more expressive and role-oriented successor to Ingress for routing traffic into the cluster.

AVOID THIS MISTAKE

Debugging the app before checking whether the request even reached it.

15 TERRAFORM WANTS TO DESTROY THE DB

DEVOPS & CLOUD DIFFICULTY ✦✦✦

THE QUESTION

terraform plan shows that a production database will be destroyed and recreated after a small change. What do you do?

SAY IT LIKE THIS (60-90 seconds, out loud)

I stop, and I don't apply. Destroy and recreate on a database means data loss. First I read the plan to see exactly which attribute forces replacement, shown as `forces replacement`. Some attributes are immutable in the provider, for example the name, engine or encryption setting. Then I decide: revert the change in code, use a supported in-place alternative, or plan a proper migration through a snapshot and restore. To guard production, I add `lifecycle { prevent_destroy = true }` on critical resources, enable deletion protection on the database itself, take a final snapshot, and require plan review in CI before apply. If a rename caused it, I use a `moved` block or `terraform state mv`, so Terraform doesn't see a brand-new resource.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`terraform plan -out=tfplan`	Review: look for `-/+` (replace) or `-` (destroy), and `# forces replacement`.
2	`terraform show tfplan`	Readable details of which attribute changed and why replacement is forced.
3	`git diff HEAD~1 -- main.tf`	Which code change triggered it? Revert or adjust.
4	`terraform state list` and `terraform state show aws_db_instance.main`	Does state match the real database and the resource address in code?
5	`moved { from = aws_db_instance.old to = aws_db_instance.main }`	Declare renames so Terraform moves state instead of recreating (or use terraform state mv).
6	`lifecycle { prevent_destroy = true }`	Makes the plan fail if anything tries to destroy this resource.
7	`aws rds create-db-snapshot --db-instance-identifier db --db-snapshot-identifier pre-change`	Take a manual snapshot before any risky change.
8	RDS deletion protection and plan approval in CI	Defence in depth: the cloud API and the pipeline both block accidents.

KEY POINTS TO MENTION

Some attribute changes can't be applied in place and force replacement. The plan is your safety net, so never apply without reading it. A renamed resource looks like destroy plus create unless you use `moved` blocks.

FIX AND PREVENTION

Revert or restructure the change, migrate data deliberately with snapshots, use `prevent_destroy` and deletion protection, keep stateful resources in separate state, and add policy checks (OPA or Sentinel) that block destroy actions on databases.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. What does -/+ mean in a plan?

A. The resource will be destroyed and recreated (replaced).

Q. What is a moved block?

A. A declarative way to tell Terraform a resource was renamed or moved, so it updates state instead of replacing the resource.

AVOID THIS MISTAKE

Applying with -auto-approve in production without reading the plan.

16 ANSIBLE PLAYBOOK FAILED ON SOME HOSTS

DEVOPS & CLOUD DIFFICULTY ✦✦✦

THE QUESTION

An Ansible playbook ran across 50 servers and failed on 8 of them. How do you handle it?

SAY IT LIKE THIS (60-90 seconds, out loud)

First I read the play recap to see exactly which hosts failed and at which task, and I read the error: connection failure, privilege problem, missing package, changed variable, or a real bug. Because Ansible is meant to be idempotent, I can usually fix the cause and re-run safely. I use the retry file or `--limit` to run only on the failed hosts. For hosts I can't reach, I check SSH, keys and sudo. Before re-running more broadly, I use `--check --diff` to preview the changes. To reduce this in future, I use `serial` and `max_fail_percentage` so a bad change stops after a small batch, `any_errors_fatal` for critical steps, `block` and `rescue` for recoverable parts, and Molecule to test roles.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`ansible-playbook site.yml (read the PLAY RECAP)`	Per-host ok, changed, failed and unreachable counts.
2	`ansible-playbook site.yml --limit @site.retry`	Re-run only on hosts that failed (when retry files are enabled).
3	`ansible-playbook site.yml -l web03 -vvv`	Verbose output on one failing host to see the full error.
4	`ansible web03 -m ping and ansible web03 -b -m command -a 'id'`	Check connectivity, SSH keys and sudo for unreachable or privilege failures.
5	`ansible-playbook site.yml --check --diff -l failed_hosts`	Preview changes before re-running.
6	`serial: 5 and max_fail_percentage: 10 (in the play)`	Roll out in batches, and stop early if too many hosts fail.
7	`block: ... rescue: ... always: ...`	Handle recoverable errors and clean up inside the playbook.
8	`molecule test`	Test roles in containers before they reach real servers.

COMMON ROOT CAUSES

SSH or key problems, sudo or privilege issues, package repositories unreachable, host-specific differences (OS version, disk space), undefined variables, tasks that aren't idempotent, timeouts, or a genuinely broken change.

FIX AND PREVENTION

Fix the per-host causes, re-run on the failed hosts, then run on all hosts again to prove convergence. Use batches (`serial`), health checks after changes, idempotent modules instead of `shell`, and CI testing.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. What does idempotent mean in Ansible?

A. Running a task repeatedly leads to the same state. Modules check the current state first and change only if needed.

Q. Why avoid the shell module?

A. It isn't idempotent by default, reports changed on every run, and is harder to make safe. Prefer dedicated modules.

AVOID THIS MISTAKE

Re-running the whole playbook on all hosts at once without understanding the failure.

17 RECOVER FROM A BAD GIT FORCE-PUSH

DEVOPS & CLOUD DIFFICULTY ✦✦✦

THE QUESTION

A teammate force-pushed to main and overwrote other people's commits. How do you recover, and how do you prevent it happening again?

SAY IT LIKE THIS (60-90 seconds, out loud)

Git rarely loses data, so I stay calm and find the old commits. The quickest sources are usually any clone that still has the old history, the pull-request merge commits, CI runs that show the old SHA, or the hosting platform's event log. In a local clone, `git reflog` shows where main pointed before. Once I have the lost commit SHA, I create a rescue branch at that commit, then restore main: either reset main to that SHA and push with `--force-with-lease`, or cherry-pick the missing commits on top, depending on what the force-push intended. Then I tell the team to fetch and reset their local branches. To prevent it: branch protection on main, disallow force pushes, require pull requests and reviews, and use `--force-with-lease` instead of `--force`.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`git reflog show origin/main (or git reflog)`	Find the SHA main pointed to before the force-push, from any clone that had it.
2	`git log --oneline --graph --all`	Compare the current history with what you expect.
3	CI and platform history	CI runs, PR merge commits and GitHub or GitLab events show old SHAs.
4	`git branch rescue/main-old OLD_SHA`	Create a branch at the old tip, so the commits are safe.
5	`git cherry-pick A..B or git merge rescue/main-old`	Re-apply the missing commits onto the current main.
6	`git push --force-with-lease origin main`	If you must restore the old history, force-push safely (it refuses if the remote changed unexpectedly).
7	Branch protection rules	Block force pushes, require PRs and status checks on main.
8	`git fetch --all && git reset --hard origin/main`	Teammates re-sync their local copies after the fix.

KEY POINTS TO MENTION

Commits aren't deleted immediately. Reflog and other clones keep them, and unreachable commits are only garbage-collected after a long time. The team should talk before rewriting shared history.

FIX AND PREVENTION

Protect main, disable force pushes, require reviews and CI, use `--force-with-lease`, and agree a rebase and merge policy. Handle it in a blameless way and educate the team.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. --force versus --force-with-lease?

A. --force overwrites the remote unconditionally. --force-with-lease refuses if someone else pushed since your last fetch.

Q. What is git reflog?

A. A local log of where HEAD and branches pointed over time. It lets you recover 'lost' commits.

AVOID THIS MISTAKE

Blaming the teammate first. Recover the commits, then fix the process.

18 TOO MANY NOISY ALERTS

DEVOPS & CLOUD DIFFICULTY ✦✦✦

THE QUESTION

Your team receives hundreds of alerts a day and has started ignoring them. How do you fix the alerting?

SAY IT LIKE THIS (60-90 seconds, out loud)

Alert fatigue is dangerous because real incidents get missed. I start by measuring: which alerts fire most, which are actionable, and which led to action. Every page should be urgent, actionable and tied to user impact. Otherwise it should be a dashboard, a ticket, or removed. I move from cause-based alerts like high CPU to symptom-based alerts built on SLOs, such as error rate and latency, and use burn-rate alerts so we're paged only when the error budget is really at risk. I tune thresholds, add `for:` durations to avoid flapping, group and deduplicate in Alertmanager, add inhibition so downstream alerts don't fire when the root cause already did, route by severity, and link a runbook in every page. Then I review alert noise weekly.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	Export the last 30 days of alerts and rank by count	Find the noisiest alerts and the ones nobody acts on.
2	`expr: rate(http_requests_total{code=~'5..'}[5m]) / rate(http_requests_total[5m]) > 0.02`	Symptom alert: error ratio above 2%, tied to user impact.
3	`for: 10m`	Alert only if the condition persists. Avoids flapping.
4	`group_by: [alertname, cluster] (Alertmanager route)`	Group related alerts into one notification.
5	`inhibit_rules: (Alertmanager)`	Mute downstream alerts when a root-cause alert is firing, such as node down.
6	`severity: page | ticket | info (labels and routes)`	Only urgent, actionable alerts wake people. Others go to tickets or chat.
7	`annotations: runbook_url: https://...`	Every alert links to what it means and the first steps to take.
8	Weekly alert review	Delete, tune or fix any alert that was ignored or needed no action.

KEY POINTS TO MENTION

Page on symptoms and error-budget burn, not every possible cause. Every page must be actionable. Use SLOs, burn-rate alerts, grouping, inhibition and runbooks. Track alert quality as a team metric, for example pages per on-call shift.

FIX AND PREVENTION

Reduce noise, define SLOs, keep dashboards for diagnosis instead of alerts, and automate safe fixes (self-healing) so humans aren't paged for known, repeatable issues.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. What is an error budget?

A. The allowed amount of unreliability under an SLO. If it is used up, you slow feature work and improve reliability.

Q. What is alert flapping?

A. An alert that repeatedly fires and resolves because a metric hovers around the threshold. Fix it with `for:` durations and hysteresis.

AVOID THIS MISTAKE

Adding more alerts after every incident without removing old ones.

19 ON-CALL AT 3 AM: INCIDENT RESPONSE

DEVOPS & CLOUD DIFFICULTY ✦✦✦

THE QUESTION

You are on call and get paged at 3 AM because the checkout service is failing. Walk me through how you handle the incident.

SAY IT LIKE THIS (60-90 seconds, out loud)

I work in a calm, structured way. First, I acknowledge the page and assess severity and user impact: is it a full or partial outage, and since when? I check dashboards and recent changes, because deployments are the most common cause. If it's serious, I declare an incident and open a channel, bring in help early rather than late, and assign roles: an incident commander, someone investigating, and someone communicating. My first goal is to mitigate, not to find the root cause: roll back the last change, fail over, scale up, or disable a feature flag. I keep a timeline as I go and post regular updates. Once service is restored, I confirm recovery from metrics, then schedule a blameless post-mortem with the root cause, impact, what went well, and action items with owners.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	Acknowledge the page and check severity	Customer impact, scope and start time. Is it getting worse?
2	Dashboards: error rate, latency, saturation, traffic	The golden signals show what changed and when.
3	`kubectl rollout history deploy/checkout and git log --since='3 hours ago'`	What changed recently: deployments, config, infrastructure?
4	`kubectl rollout undo deploy/checkout`	Fast mitigation: roll back the last deployment if it correlates.
5	`kubectl logs deploy/checkout --since=15m | grep -i error`	A quick look at the errors for clues.
6	Open an incident channel and assign roles	Commander, investigator, communicator. Don't investigate alone for long.
7	Status updates every 15 to 30 minutes	State what is known, what is being done, and when the next update comes.
8	After recovery: post-mortem within a few days	Timeline, root cause, impact, action items with owners. Blameless.

KEY POINTS TO MENTION

Mitigate first, investigate later. Communication matters as much as technical work. Use runbooks and keep a timeline. Escalate early. A blameless culture improves reliability.

FIX AND PREVENTION

Improve runbooks, alerts and rollback automation after each incident. Track MTTR, follow up on actions, and rotate on-call fairly with good handover notes.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. What does MTTR mean?

A. Mean time to recovery (or restore): how long incidents take to resolve on average.

Q. What makes a post-mortem blameless?

A. It focuses on systems and processes, not individuals, so people report problems honestly and the real causes get fixed.

AVOID THIS MISTAKE

Trying to be a hero alone for an hour before asking for help.

20 DATABASE FAILOVER AND REPLICATION LAG

DEVOPS & CLOUD DIFFICULTY ✦✦✦

THE QUESTION

Your primary database server failed. How should failover work, and how do you deal with replication lag?

SAY IT LIKE THIS (60-90 seconds, out loud)

A good design has automated, tested failover. Typically there's a primary and one or more replicas in other availability zones, using a managed service such as RDS Multi-AZ or Aurora, or a tool like Patroni for Postgres. When the primary fails, a replica is promoted, and applications reconnect through a DNS name, proxy or virtual IP, so they need no config change. The risk is replication lag: with asynchronous replication the replica may be missing the latest transactions, so we can lose data up to the lag at the time of failover. I monitor lag, use synchronous or semi-synchronous replication for critical data, and decide what's acceptable from the RPO. I also prevent split-brain with fencing and quorum, and run failover drills regularly.

#	COMMAND / CHECK	WHAT IT TELLS YOU
1	`SELECT now() - pg_last_xact_replay_timestamp(); (Postgres replica)`	Replication lag as time. Also check pg_stat_replication on the primary.
2	`SHOW REPLICA STATUS\G (MySQL)`	Seconds_Behind_Source, IO and SQL thread states, and errors.
3	`patronictl list (Patroni)`	Cluster state: leader, replicas, lag and timeline.
4	`aws rds describe-db-instances --query 'DBInstances[].[DBInstanceIdentifier,MultiAZ,DBInstanceStatus]'`	Is Multi-AZ enabled, and what is the state?
5	`aws rds reboot-db-instance --db-instance-identifier db --force-failover`	Test a Multi-AZ failover in a controlled drill.
6	`SELECT pg_promote(); (Postgres replica)`	Promote a Postgres replica to primary (manual failover).
7	**Endpoint: DNS name, proxy (RDS Proxy, PgBouncer) or VIP**	Applications connect through an endpoint that moves to the new primary.
8	**Fencing and quorum**	Make sure the old primary cannot accept writes after failover, which prevents split-brain.

COMMON ROOT CAUSES

Primary hardware or AZ failure, long-running queries or heavy writes causing lag, network latency between sites, single-threaded replica apply, disk saturation on replicas, misconfiguration after failover.

FIX AND PREVENTION

Use managed or orchestrated HA, give replicas similar capacity to the primary, monitor lag with alerts, test failover regularly, and match replication mode and backups to RPO and RTO. Keep point-in-time recovery as a backstop.

FOLLOW-UP QUESTIONS THEY MAY ASK

Q. Synchronous versus asynchronous replication?

A. Synchronous waits for the replica to confirm, so no data loss but higher write latency. Asynchronous is faster but can lose recent transactions on failover.

Q. What is split-brain?

A. Two nodes both believe they are primary and accept writes, causing conflicting data. It is prevented with quorum and fencing.

AVOID THIS MISTAKE

Assuming failover works because it is configured. Test it.

21 RAPID-FIRE ROUND

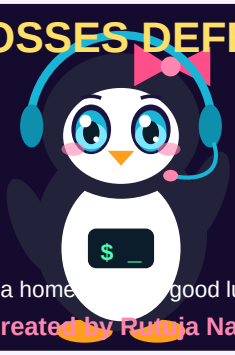
10 quick questions, 1 to 2 sentence answers

1	**Layer 4 versus layer 7 load balancing?**	L4 balances on IP and port (TCP or UDP) and is very fast. L7 understands HTTP, so it can route by host, path or header and terminate TLS.
2	**What is a DaemonSet?**	A Kubernetes object that runs one pod on every node (or selected nodes), used for log collectors, monitoring agents and CNI.
3	**Helm versus Kustomize?**	Helm templates and packages apps as charts with values. Kustomize patches plain YAML with overlays, with no templating.
4	**What is a Terraform module?**	A reusable group of resources with inputs and outputs, so you define a pattern once and reuse it across environments.
5	**What is GitOps?**	Git is the source of truth for desired state. A controller such as Argo CD or Flux continuously applies it to the cluster and reverts drift.
6	**What are Linux capabilities?**	The power of root split into small privileges, like binding low ports or changing the clock, so a process gets only what it needs.
7	**What is a CRD and an operator?**	A CRD adds a new resource type to Kubernetes. An operator is a controller that manages that resource's lifecycle, like a database cluster.
8	**MTTR versus MTBF?**	MTTR: mean time to recover from a failure. MTBF: mean time between failures. Good teams reduce MTTR and increase MTBF.
9	**Explain the TLS handshake briefly.**	The client and server agree on a version and cipher, the server proves its identity with a certificate, and they derive shared session keys for encryption.
10	**What does set -euo pipefail do?**	Stops the script on errors (-e), on unset variables (-u), and when any command in a pipeline fails (pipefail).

BEFORE THE INTERVIEW: FINAL CHECKLIST

- Practise each answer out loud in 60 to 90 seconds. Record yourself once.
- Prepare a real example for each topic: "Last time I saw this, I..."
- Know why a command works, not just its syntax.
- Admit what you don't know, then explain how you would find out.
- End answers with prevention: monitoring, automation, documentation.
- Keep a home lab (VM, Docker, minikube). Break things on purpose and fix them.

ALL BOSSES DEFEATED!



Practice out loud, build a home good luck with your interviews.

Created by Rutuja Naik